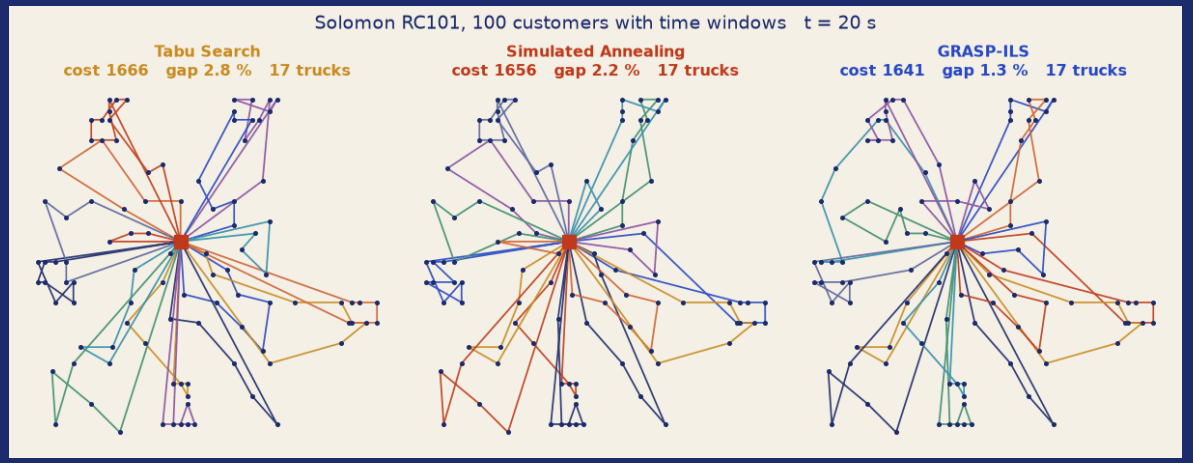
Routing deliveries when the clock matters

From a model with hourly traffic to three metaheuristics tested from 100 to 10 000 customers and on real city road networks



Live from our solvers: same instance, same 20 s

Pablo · Dag · Farida
13 November 2026

THE QUESTION

One depot, a fleet, customers with windows, and traffic that changes every hour

“Which truck serves which customers, in which order, and when does it leave, so that the total route duration is minimal?”

Level B: CVRP (capacity)

Level A: CVRPTW (capacity and time windows)

Our extension: hourly traffic

Q — **Capacity**

every truck carries at most Q units

[e, l] — **Time window**

service starts inside it; waiting is allowed

ρ(h) — **Traffic**

a trip leaving at hour h takes ρ(h) times its free flow time

z — **Objective**

total time from leaving the depot to coming back, all trucks

Our first objective could not see the clock

Version 1: total travel time on fixed arcs

$$\min \; z = \sum_{(i,j) \in A} c_{ij}\, x_{ij}$$

c_ij is one number per pair of places, the same at 8:00 and at 22:00.

What the review told us

One objective, one component, exactly what the brief asks. Every symbol defined. Traffic inside the model, not as a footnote.

01

The hour is invisible

With a fixed c_ij, leaving in the peak or at night gives the same z: traffic, the point of the ADEME call, had no effect.

02

It is only distance

At constant speed the sum is the distance: we were solving the textbook CVRP, not the brief, which says total route duration.

03

Waiting was free

A truck waiting 40 min for a window to open cost nothing, yet the driver and the truck are paid for that time.

We switched to the total route duration, with a traffic matrix per hour

Objective, one component

$$\min \; z = \sum_{k \in K} \left(t_{n+1}^{k} - t_{0}^{k} \right)$$

back at the depot minus left the depot, added over the trucks: traffic and waiting are inside z

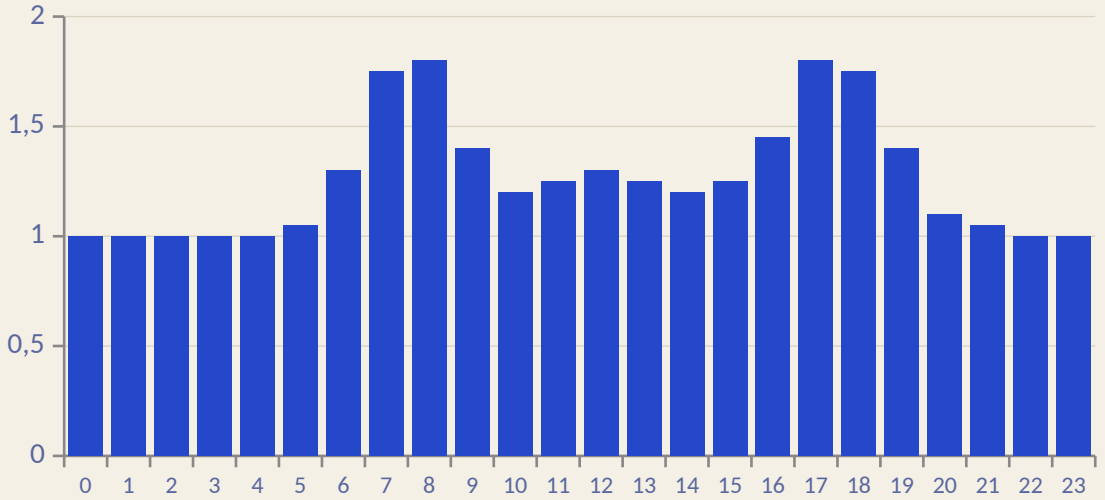
Traffic matrix and arrival time

$$c_{ij}^{h} = \rho_h \, c_{ij}$$

$$a_j = t_i + s_i + \sum_h c_{ij}^{h} \, y_{ih} \qquad \text{if truck goes } i \to j$$

y_ih = 1 picks the hour the truck leaves i; big-M switches the line off for unused arcs

Traffic factor ρ(h) by hour of departure



×1.80

peak at 8:00 and 17:00

247.2 min

toy optimum: ILP = full enumeration

The problem is NP-hard, and the exact model stops at a few customers

Hamiltonian cycle
NP-complete (Karp 1972)

→

TSP
1 truck, no windows, ρ = 1

→

Our problem
capacity, windows, traffic

A graph has a Hamiltonian cycle if and only if our instance with K = 1 and arc times 1/(m+1) or 2/(m+1) has a plan of duration at most 1. Built in $O(m^2)$: the decision version is NP-complete.

Consequence: no polynomial exact algorithm is expected; we measure where the ILP (HiGHS, 60 s) stops.

7 to 8 customers: full model with hourly traffic, proven optimal

100 customers: windows, no traffic, on C101 and R101 (= best known)

< 50 customers on mixed RC101: 6 to 13 % gap at 60 s

Three metaheuristics, one evaluator, one protocol

Tabu Search

Pablo

STEP

Best of 200 sampled moves each step, even if worse

GUIDANCE

Moved customer is tabu for ~20 iterations; aspiration on a new best

ENGINE

Start: time oriented nearest neighbor

TUNED

tenure 20, 200 candidates, 15 neighbors

Simulated Annealing

Dag

STEP

One random move; accept a worse one with probability e^(−Δ/T)

GUIDANCE

Temperature cools geometrically over the time budget

ENGINE

Start: randomized cheapest insertion; compiled (numba)

TUNED

T_0 factor 0.005 (0.01 in the scale study), T_f 0.005

GRASP-ILS

Farida

STEP

Randomized regret insertion, then a granular descent (6 neighborhoods)

GUIDANCE

90 kicks per iteration: remove 10 to 20 % of customers, reinsert, keep if better

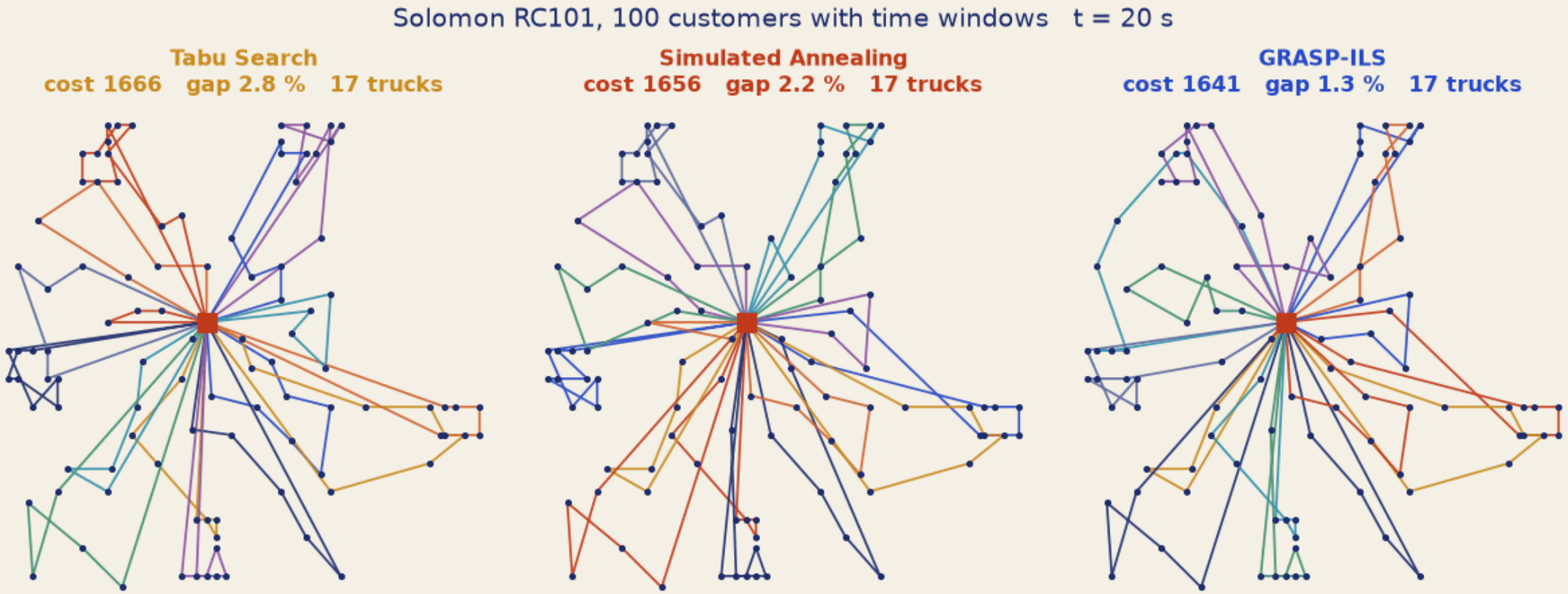
ENGINE

Time windows checked in O(1) per move

TUNED

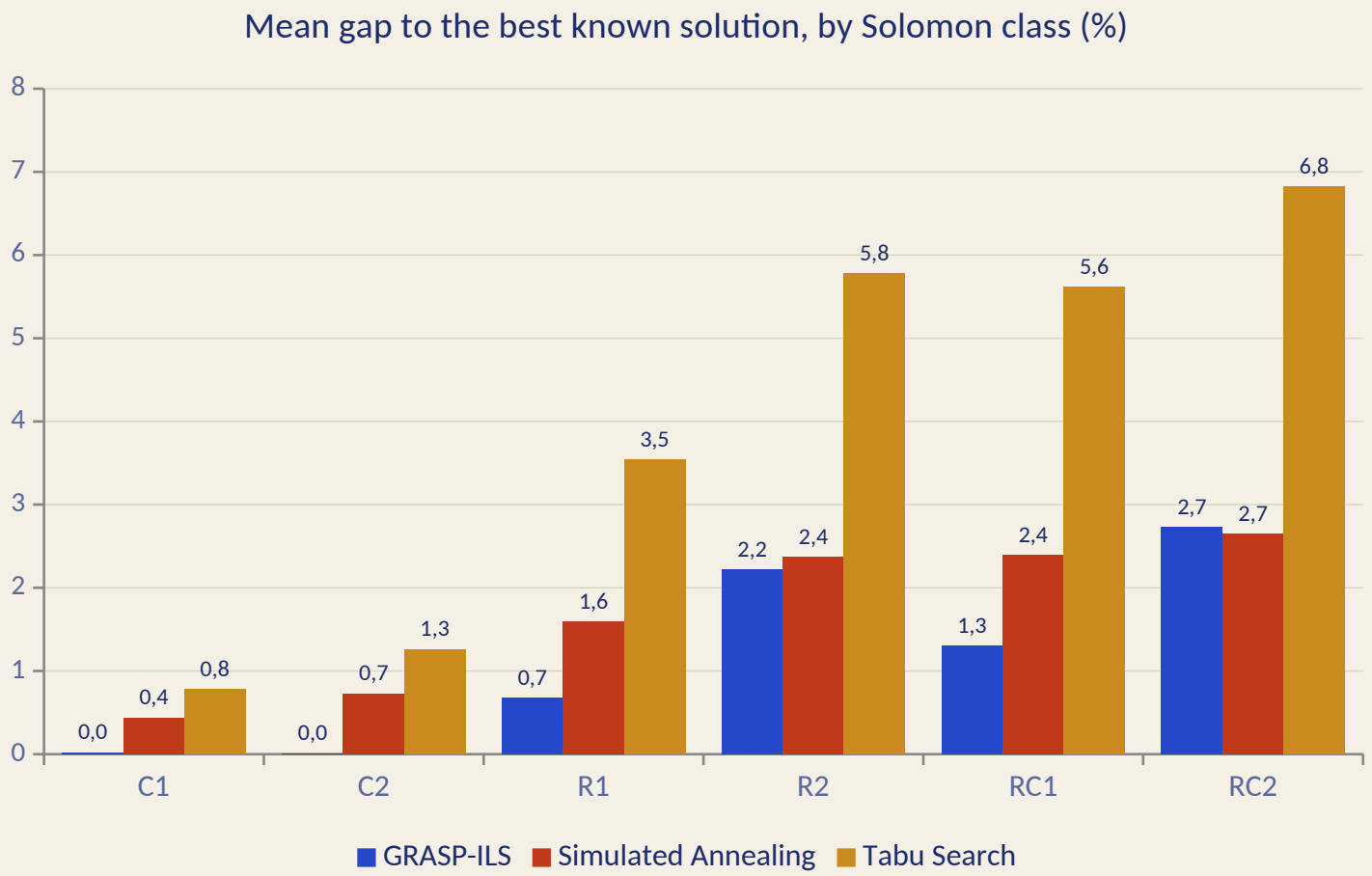
α 0.3, first improvement, 90 kicks

Same 100 customers, same 20 seconds: three ways to search



Each frame is the best feasible plan the solver returned after t seconds (seed 1), checked by the shared evaluator. Colors are trucks; the red square is the depot.

Round 1: GRASP-ILS wins on the 50 Solomon test instances



1.16 %
GRASP-ILS
worst 9.8 %, at best known
162 of 500

1.70 %
Simulated Annealing
worst 6.5 %, at best known
106 of 500

3.98 %
Tabu Search
worst 14.0 %, at best known
110 of 500

Friedman $p = 4 \cdot 10^{-13}$; every pair differs (Wilcoxon, Holm, 5 %). GRASP-ILS is chosen: feasible on every run, lowest mean gap.

“100 customers on Euclidean points is not the real world. Show me the limit of each algorithm.”

10 000
customers at least

Roads
real networks, not straight lines

2 countries
same protocol, explain the difference

Parameters
sensitivity of each algorithm

Night
what changes outside the peak

Your opinion
which algorithm, in which scenario

NEW TEST BED

The instance problem: we needed bigger, real, and still measurable data

Data	Customers	Windows	Reference for quality	Why
Solomon (round 1)	100	yes	best known	tuning and statistics
Homberger R1	400 · 800 · 1 000	yes	best known	how quality breaks with n
Synthetic uniform	1 000 to 10 000	no	best run, radial bound	the limit in time and memory
Paris (France)	999 real	no	best of all runs	road network, real asymmetry
São Paulo (Brazil)	999 real	no	best of all runs	second country, same protocol

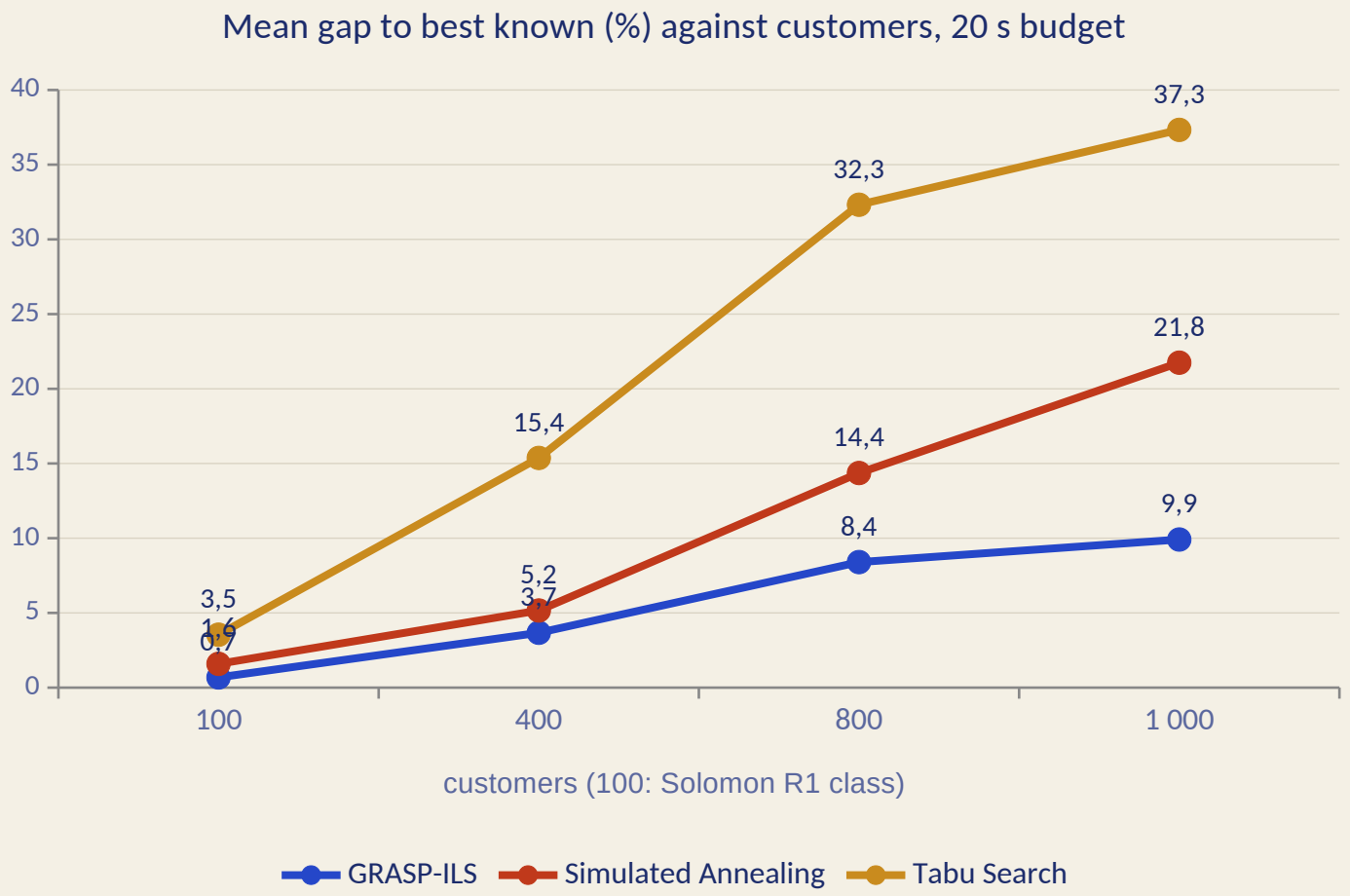
Road networks: OpenStreetMap through the RRNCO dataset; travel distances and free flow times are shortest paths on the real graph.

Two cities, measured

	Paris	S. Paulo
mean speed	21 km/h	32 km/h
circuity (road / straight)	1.51	1.65
pairs with A→B ≠ B→A	97 %	99 %
area covered	8.9 km²	8.7 km²

Same area, same number of customers: differences come from the roads.

As n grows, every algorithm loses quality, but not at the same speed



GRASP-ILS

9.9 % at 1 000: it finishes a single iteration in 20 s, yet its regret construction alone beats the others.

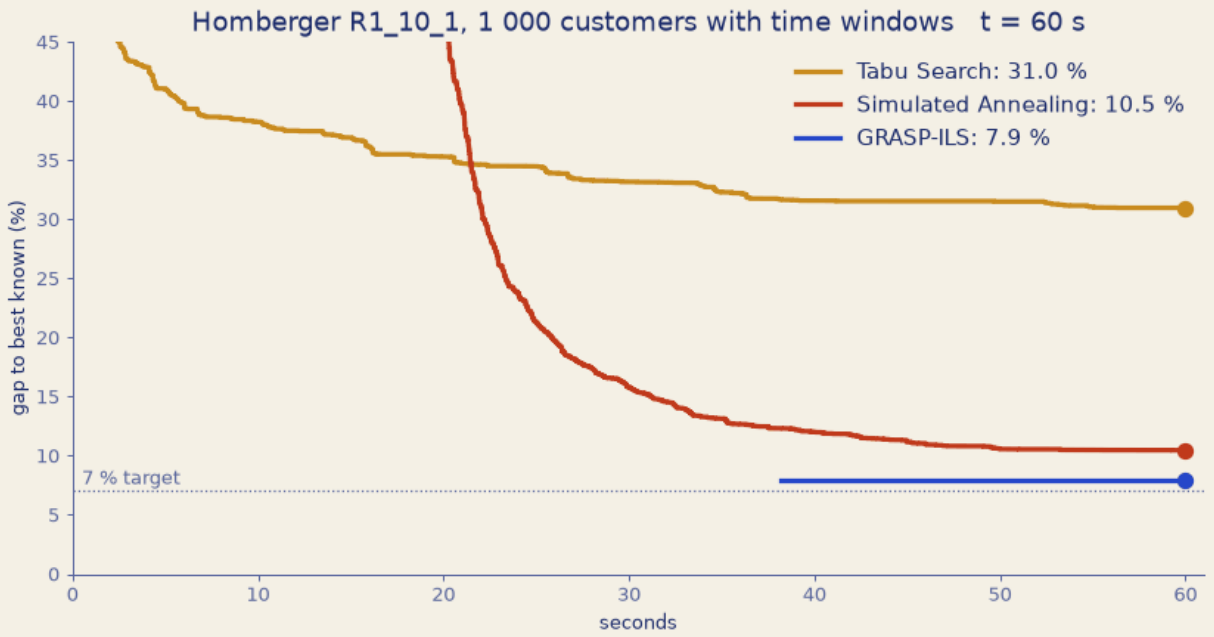
Simulated Annealing

21.8 % at 1 000: 16 million moves are not enough: with 6 times the budget its gap is halved (next slide).

Tabu Search

37.3 % at 1 000: 175 trucks against 107 for GRASP-ILS: its gap is mostly extra trucks that its moves rarely remove.

More time helps SA most; GRASP-ILS stays ahead at every budget



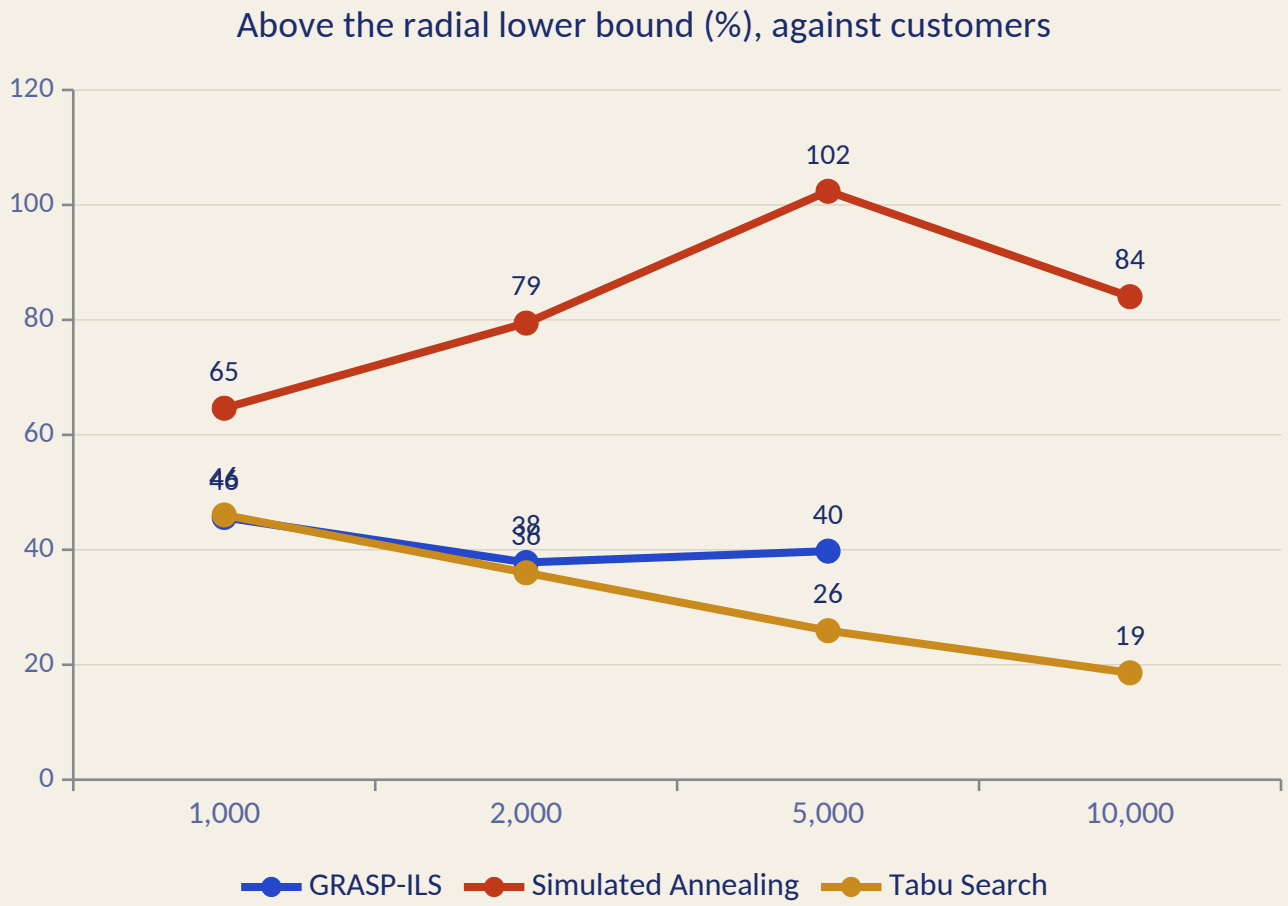
Gap at 1 000	20 s	60 s	120 s
GRASP-ILS	9.9 %	9.0 %	7.8 %
Simulated Annealing	21.8 %	15.0 %	10.4 %
Tabu Search	37.3 %	33.8 %	31.7 %

÷2.1

SA gap from 20 s to 120 s at 1 000 customers: it was starved of time, not of ideas

Within the 7 % target: GRASP-ILS up to 800 customers with 120 s (6.7 %); at 1 000 it needs more than 120 s (7.8 %).

From 1 000 to 10 000 customers: who still answers?



GRASP-ILS: no plan at 10 000

Budget 60 s. One iteration takes it all: 63 s at 2 000, 125 s at 5 000, more than 684 s at 10 000 (budget 300 s).

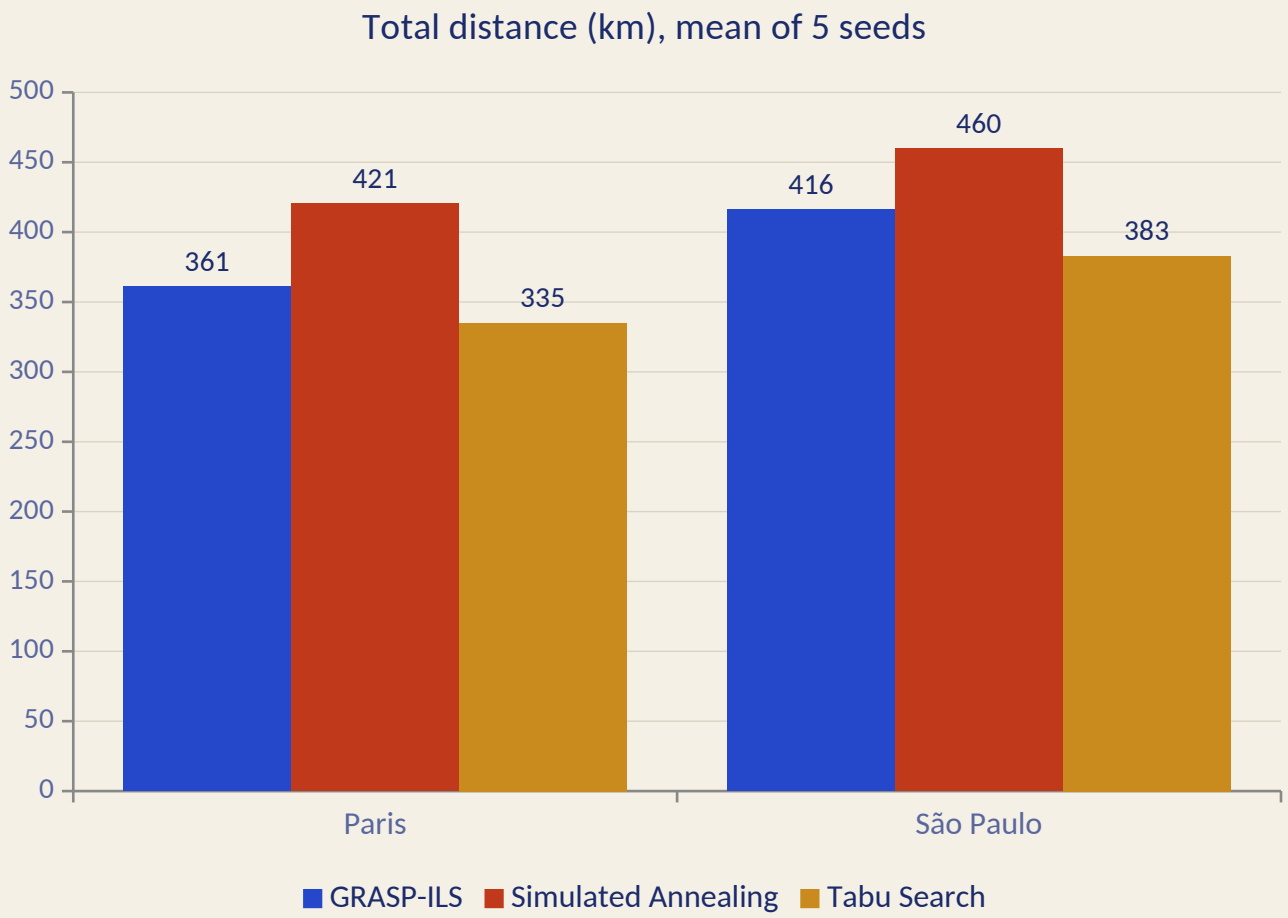
Tabu Search: 18.6 % at 10 000

Time per iteration barely grows: 0.7 ms at 1 000, 2.1 ms at 10 000, thanks to its neighbor lists.

Simulated Annealing: 84 % at 10 000

Moves per second fall with n (1.0 million/s at 1 000, 0.16 at 5 000) and its cooling ends too early.

On real roads without windows, Tabu Search wins: the opposite of Solomon



No windows, so construction matters less

Capacity alone: every truck can take any customer. What counts is how many improving moves are made: Tabu makes about 30 000 iterations in 60 s, GRASP-ILS one.

GRASP-ILS never reaches its kicks

With 0, 10 or 30 kicks the result is identical (357.5 km in Paris): the first descent fills the 60 s.

Five times the time does not change the order

300 s: Tabu 318 km, GRASP-ILS 345 km in Paris. Tabu at 60 s is 5.5 % above its own 300 s plan.

At night the same deliveries take 44.6 % less time: that is the traffic factor



Paris

−44.6 %

1,831 min at 08:00 → 1,014 min at 22:00

São Paulo

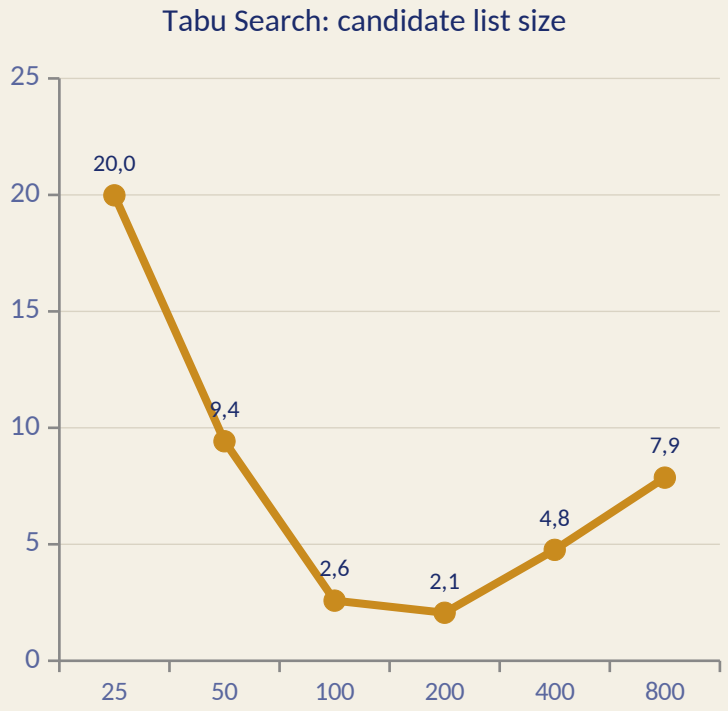
−43.7 %

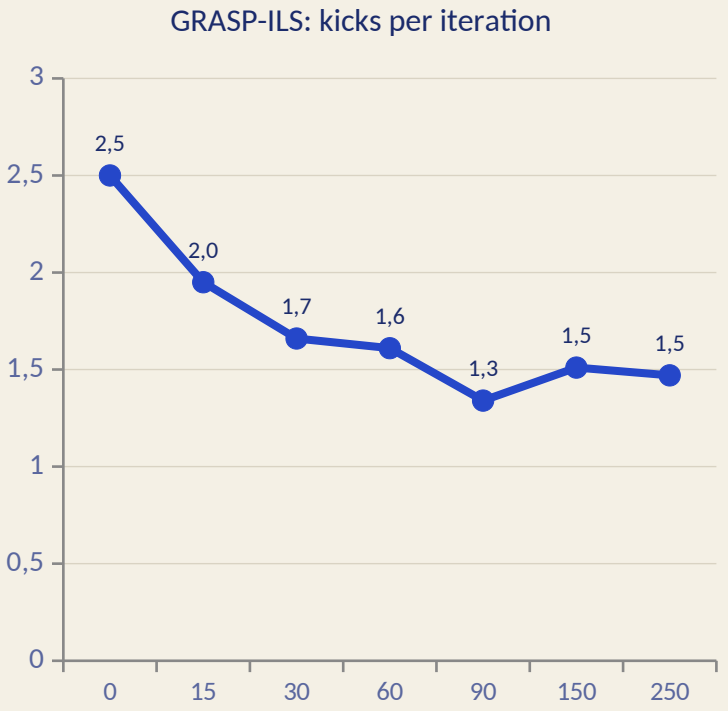
1,511 min at 08:00 → 850 min at 22:00

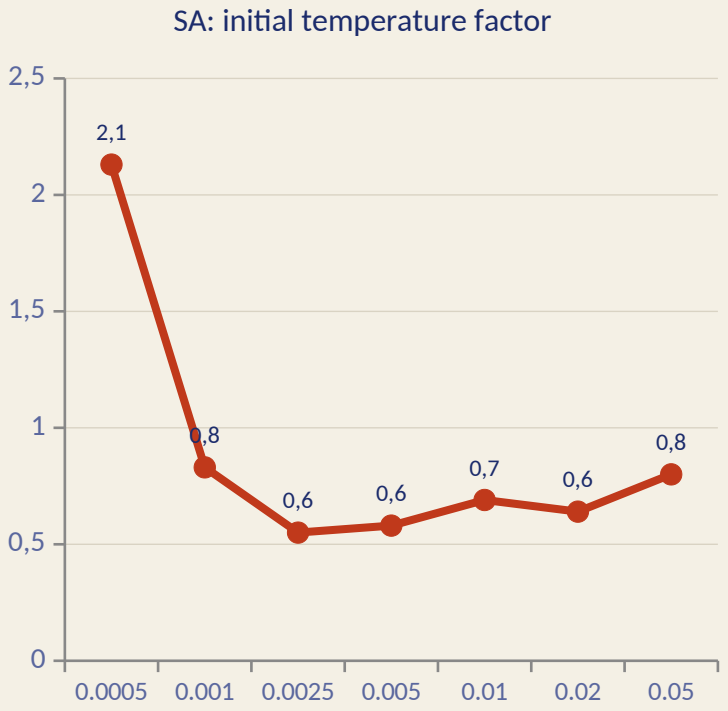
Straight segments join consecutive stops; travel times are road network times times ρ(h).

1 − 1/ρ(8) = 1 − 1/1.80 = 44.4 %. Routes last 32 min on average: only 1 of 57 changes hour, none crosses midnight. The night measures the size of the traffic effect; a test where the order of visits must change needs routes spanning several hours.

Each algorithm has one parameter that can break it







Tabu Search is the most fragile: 25 candidates 20.0 %, 800 7.9 %, tuned 200 2.1 %. GRASP-ILS without kicks doubles its gap (2.5 % against 1.3 %). SA is flat from 0.001 to 0.05 and breaks only near $T_0 = 0$ (2.1 %): it becomes a descent.

Our choice: GRASP-ILS for the ADEME pilot, with a rule to switch

Final choice **GRASP-ILS** the brief is the CVRPTW: time windows, a depot, a few hundred deliveries. Switch to Tabu Search above 2 000 customers or without windows.

Scenario	Choose	Because (our data)
About 100 customers, time windows	**GRASP-ILS**	1.16 % mean gap, best known on 162 of 500 runs; significantly ahead of SA and Tabu Search
One bad run is not acceptable (100 customers)	**Simulated Annealing**	worst run 6.5 %, every run within 7 %; slightly ahead on wide windows (RC2)
400 to 1 000 customers, time windows	**GRASP-ILS**	lowest gap at every size and budget; within 7 % up to 800 customers with 120 s (6.7 %)
About 1 000 customers on a real road network, no windows	**Tabu Search**	6 % to 7 % from the best plan found, against 15 % to 17 % for GRASP-ILS: 30 000 iterations against 1
10 000 customers, no windows	**Tabu Search**	the only good plan: 18.6 % above a lower bound, minimum trucks; GRASP-ILS returns nothing
Peak or night departure	**Tabu Search**	as without traffic: the ranking does not change with the hour; the 44.6 % night saving is the traffic factor

Caveat: windows were tested up to 1 000 customers, no windows from 1 000 to 10 000; we did not test 10 000 customers with windows.

TRANSPARENCY

What we did not solve, and what we would do next

Equal time is not equal code

SA is compiled (numba); Tabu Search and GRASP-ILS are pure Python. Compiling the GRASP-ILS descent is the first step to scale it.

Traffic is assumed

ρ(h) is our profile, not measured data; with measured speeds no equation changes, only the matrix.

France against Brazil, not Egypt

Open road matrices existed for Paris and São Paulo; Cairo needs an OSM extraction we did not validate in time.

One departure time per fleet

The model lets each truck choose its departure; the metaheuristics fix one. A 1-D search per route would close the gap.

First in, first out

A later departure can arrive earlier when the hour changes; cutting trips at the hour removes it.

Our solver overran its budget

GRASP-ILS reads its clock only between iterations: 684 s on a 300 s budget at 10 000. A defect of our code, fixed by checking time inside the construction.

IN ONE SLIDE

Our choice is GRASP-ILS for the ADEME pilot. When the city grows past 2 000 customers or has no windows, Tabu Search takes over

1.16 %

GRASP-ILS mean gap on 500 Solomon runs, chosen method

7.8 %

GRASP-ILS at 1 000 customers with time windows, 120 s

18.6 %

Tabu Search above a lower bound at 10 000 customers, the only method that scales

Thank you. Questions?